

Aufgabe 1.1 (4 Punkte)(A)

Angenommen Sie haben einen Rechner, der maximal 100.000 Rechenschritte pro Algorithmus ausführen kann. Auf diesem Rechner wollen Sie verschiedene Algorithmen mit unterschiedlichen Laufzeiten testen. Berechnen Sie für jeden Algorithmus, wie lang die Eingabe maximal sein darf, damit der Algorithmus in 100.000 Rechenschritten stoppt. Berechnen sie außerdem, wie lang die Eingabe maximal sein darf, wenn der Rechner stattdessen 1.000.000 Rechenschritte pro Algorithmus ausführen darf.

	n	$10^4 \cdot n^2$	2^n	$n!$
max Eingabe mit 10^5	10^5	$\lfloor \sqrt{10} \rfloor = 3$	$\lfloor \log_2 10^5 \rfloor = \lfloor 5 \cdot \log_2 10 \rfloor = 16$	8
max Eingabe mit 10^6	10^6	10	$\lfloor \log_2 10^6 \rfloor = \lfloor 6 \cdot \log_2 10 \rfloor = 19$	9

für $n!$ wird folgendes java Programm genutzt

```
public static int fac(int n){
    if(n == 0 || n == 1){
        return 1;
    }
    return n*fac(n-1);
}
public static void main(String[] args){
    while(fac(i) < 1000000){
        System.out.println("Eingabelänge: "+i+" Rechenschritte "+fac(i));
        i+=1;
    }
    System.out.println("geht nicht mehr->Eingabelänge: "+i+" Rechenschritte "+fac(i));
}
```

das folgende Ausgabe Erzeugt

```
Eingabelänge: 0 Rechenschritte 1
Eingabelänge: 1 Rechenschritte 1
Eingabelänge: 2 Rechenschritte 2
Eingabelänge: 3 Rechenschritte 6
Eingabelänge: 4 Rechenschritte 24
Eingabelänge: 5 Rechenschritte 120
Eingabelänge: 6 Rechenschritte 720
Eingabelänge: 7 Rechenschritte 5040
Eingabelänge: 8 Rechenschritte 40320
```

Eingabelänge: 9 Rechenschritte 362880

geht nicht mehr->Eingabelänge: 10 Rechenschritte 3628800

Aufgabe 1.4 (4 Punkte)(A)

Folgender Algorithmus überprüft, ob in einem gegebenen binären Array nur Nullen eingetragen sind:

Require: Binäres Array $a[1, \dots, n]$
for $i = 1$ **to** n **do**
 if $a[i] \neq 0$ **then return** False
return True

Wie viele Einträge des Arrays müssen im worst-case, average-case, sowie best-case überprüft werden? Begründen Sie jeweils Ihre Antwort.

Hinweis: Verwenden Sie die geometrische Summenformel.

Worst Case

Die gesamte For-Schleife muss durchlaufen werden dies ist der fall, wenn das Array nur Nullen enthält und wenn das Array bis auf die letzte Stelle nur Nullen enthält

$$\rightarrow f(n) = c_1 n \in \mathcal{O}(n)$$

Best Case

Im Best Case ist gleich der erste Eintrag ungleich Null und somit wird direkt das return Statement getriggert

$$\rightarrow f(n) = c_1 \in \mathcal{O}(1)$$

Average Case

Idee

Die Arrays mit gleichen Laufzeiten fallen in folgende Einteilung

Bsp.:

$n=5$

Form	Anzahl Möglichkeiten	Position der ersten nicht Null stelle	Aufwand
[1xxxx]	2^4	1	1
[01xxx]	2^3	2	2
[001xx]	2^2	3	3
[0001x]	2^1	4	4
[00001]	2^0	5	5

Form	Anzahl Möglichkeiten	Position der ersten nicht Null stelle	Aufwand
[00000]	1	6	5

Auf basis dieser Erkenntnis definiert man die Zufallsgröße X_j wie folgt

$$X_j = \begin{cases} j, & \text{für } j \leq n \\ n, & \text{für } j = n + 1 \end{cases}$$

Wobei j die Position der ersten Stelle im Array angibt, die nicht Null ist. Aus dieser Zufallsgröße und der zugehörigen Anzahl der Möglichkeiten folgt die zugehörige Wahrscheinlichkeit

$$p_j = \begin{cases} \frac{2^{(n-j)}}{2^n}, & \text{für } j \leq n \\ \frac{1}{2^n}, & \text{für } j = n + 1 \end{cases}$$

Der Erwartungswert von X ist also

$$\begin{aligned} E(X) &= \sum_{i=1}^{n+1} x_i \cdot p_{n+1} = \\ &= x_{n+1} \cdot p_{n+1} + \sum_{i=1}^n x_i \cdot p_i = \\ &= n \frac{1}{2^n} + \sum_{i=1}^n i \frac{2^{n-i}}{2^n} = \\ &= \left(\frac{n}{2^n} + \sum_{i=1}^n i \frac{2^n}{2^i 2^n} \right) = \\ &= \left(\frac{n}{2^n} + \sum_{i=1}^n i 2^{-i} \right) = \\ &= \left(\frac{n}{2^n} + \sum_{i=1}^n i \left(\frac{1}{2} \right)^i \right) \end{aligned}$$

Die geometrische Summenformel $\sum_{i=1}^n x^i = \frac{x(x^n-1)}{x-1}$ reicht nicht aus. Um die summe aufzulösen setzen wir $f(x) = \sum_{i=1}^n i x^i$ Durch folgende Zusammenhänge lässt sich die Summe lösen

$$\begin{aligned} \sum_{i=1}^n i x^i &= x \frac{d}{dx} \left(\sum_{i=1}^n x^i \right) = \\ &= x \frac{d}{dx} \left(\frac{x(x^n-1)}{x-1} \right) = \\ &= \frac{x(n x^{n+1} - (n+1)x^n + 1)}{(x-1)^2} \end{aligned}$$

mit $x := \frac{1}{2}$ folgt

$$\begin{aligned} \sum_{i=1}^n i \left(\frac{1}{2} \right)^i &= \frac{\frac{1}{2} \left(n \left(\frac{1}{2} \right)^{n+1} - (n+1) \left(\frac{1}{2} \right)^n + 1 \right)}{\left(\frac{1}{2} - 1 \right)^2} = \\ &= (1 + 2^{-1-n}n - 2^{-n}(1+n))2 \end{aligned}$$

Durch Einsetzen erhält man

$$\begin{aligned}
 & \left(\frac{n}{2^n} + \sum_{i=1}^n i \left(\frac{1}{2} \right)^i \right) = \\
 &= \frac{n}{2^n} + (1 + 2^{-1-n}n - 2^{-n}(1+n))2 = \\
 &= \frac{n}{2^n} + 2 + 2^{-n}n - 2^{-n+1}(1+n) = \\
 &= \frac{n}{2^n} + 2 + 2^{-n}n - 2^{-n+1} - 2^{-n+1}n = \\
 &= \frac{n}{2^n} + 2 + n \left(\frac{1}{2} \right)^n - \left(\frac{1}{2} \right)^{n-1} - n \left(\frac{1}{2} \right)^{n-1} \\
 &= \frac{n}{2^n} + 2 - \left(\frac{1}{2} \right)^{n-1} + n \left(\frac{1}{2} \right) \left(\frac{1}{2} \right)^{n-1} - n \left(\frac{1}{2} \right)^{n-1} \\
 &= \frac{n}{2^n} + 2 - \left(\frac{1}{2} \right)^{n-1} + n \left(\frac{1}{2} \right)^{n-1} \left(\frac{1}{2} - 1 \right) \\
 &= \frac{n}{2^n} + 2 - \left(\frac{1}{2} \right)^{n-1} + n \left(\frac{1}{2} \right)^{n-1} \left(-\frac{1}{2} \right) \\
 &= n \left(\frac{1}{2} \right)^n + 2 - \left(\frac{1}{2} \right)^{n-1} - n \left(\frac{1}{2} \right)^n \\
 &= 2 - \left(\frac{1}{2} \right)^{n-1}
 \end{aligned}$$

Die Laufzeitfunktion ist also

$$f(n) = c_1 \left(2 - \left(\frac{1}{2} \right)^{n-1} \right) \in \mathcal{O}(1)$$